How Cass Works

There are a few major parts that make Cass work, with minor parts that go along with them. These are discussed below.

Two Interfaces

Cass has two interfaces that work nearly identically. Depending on your coding style, you can choose the one that you prefer. One uses `.get_...` methods to get objects, while the other prefers constructors to create objects. Both are equally good. As an example, both `cass.get_summoner(puuid="...", region="NA")` and `Summoner(puuid="...", region="NA")` work exactly the same.

Settings

There are a few settings in Cass that should be modified, and more that can be modified. See Settings for more info.

Ghost Loading

A *ghost object* is an object that can be instantiated without all of its data. It is therefore a shadow of itself, or a *ghost*. Ghost objects know how to load the rest of their data using what they were given at init. This is what allows you to write `a_summoner = Summoner(puuid="...", region="NA")` followed by `a_summoner.level`. The latter will trigger a call to the data pipeline (discussed below) to pull the rest of the data for `a_summoner` by using `a_summoner.puuid`.

Most top-level objects in Cass are ghost objects and therefore know how to load their own data.

For developers who are interested, the implementation for ghost objects can be found in our `merakicommons` repository on GitHub.

Data Pipeline

The data pipeline is the series of caches, databases, and data sources (such as the Riot API) that both provide and store data. Data sources provide data, while data sinks store data; we call both of these "data stores". Some parts of the data pipeline are only data sources (for example, the Riot API), while others are both data sources and data sinks (for example, caches and databases). The data pipeline is a list of data stores, where the order the data stores specifies how data is pulled and stored (see the next paragraph). Usually faster data stores go at the beginning of the data pipeline.

When data is queried, a query dictionary is constructed containing the information needed to uniquely identify an object in a data source (e.g. a `region` and `summoner.id` are required when querying for `Summoner` objects). This query is passed up the data pipeline through the data sources, and at each data source the data pipeline asks if that source can supply the requested object. If the source can supply the object (for example, if the object is in the database, or if the Riot API can send the object/data), it is returned. If the source does not supply the object, the next data source in the pipeline is queried. If no data source can provide an object for the query, a `datapipelines.NotFoundError` is thrown.

After an object is returned by a data source, the object gets passed back down the pipeline. Any data sinks along the way store the object that was returned by the data source. In this way, the cache (which should be at the front of the data pipeline) will store any object that a database or the Riot API returned.

A data pipeline containing an in-memory cache and the Riot API is created by default. The pipeline can be accessed via `settings.pipeline`, although users should rarely if ever touch this object after it has been instantiated.

See Data Pipeline for more details.

Searchable Containers

Most lists, dictionaries, and sets (all of which are containers) can be searched by most values that make sense. For example, the below line of code finds the first game in which `Teemo` was played in the match history of the specified summoner (note that all participants in the match are searched, not just the specific summoner for whom the match history was pulled).

```
a_teemo_game = Summoner(puuid="...", region="NA").match_history["Teemo"]
```

You can also search using objects rather than strings:

```
all_champions = Champions(region="NA")
teemo = all_champions["Teemo"]
a_teemo_game = Summoner(puuid="...", region="NA").match_history[teemo]
```

All matches in a summoner's match history where `Teemo` was in the game can be found by using `.search` rather than the `[...]` syntax:

```
# We will truncate the summoner's match history so we don't pull thousands of matche
match_history = Summoner(puuid="...", region="NA").match_history(begin_time=Patch.fr
all_teemo_games = match_history.search("Teemo")
```

You can also index on items in a match. For example:

```
...match_history["Sightstone"]
```

will find a game in the summoner's match history where someone ended the game with a Sightstone (or Ruby Sightstone) in their inventory.

Below is a final (very convenient) snippit that allows you to get your participant in a match:

```
me = Summoner(puuid="...", region="NA")
match = me.match_history[0]
champion_played = match.participants[me].champion
```

Searchable containers are extremely powerful and are one of the reasons why writing code using Cass is both fun and intuitive.

Match Histories Work Slightly Differently

The match history of a summoner is handled slightly differently than most objects in Cass. M
importantly, it is not Cached or stored in databases we create. This is largely because the logic for

doing so is non-trivial, and we haven't implemented it yet – although we hope to. Therefore match histories are requested from the Riot API every time the method is called. You are encouraged to cache the results yourself if you wish.

Match histories are also lazily loaded.